Продвинутое машинное обучение

Лекция 6: Подбор гиперпараметров и регуляризация: случайный и байесовский поиск, Hyperband / ASHA, early stopping, Optuna, трекинг экспериментов

Автор курса: Лапшина Анастасия (University of Maryland, College Park, ВШЭ)

НИУ ВШЭ 2026-2027

Часть 1. Задача HPO: почему это дорого и как не перебирать всё

Что мы оптимизируем?

Гиперпараметры λ задаются до обучения и не подбираются градиентом. Нужен набор, минимизирующий ошибку на отложенных данных:

$$\lambda^* = \arg\min_{\lambda \in \Lambda} \mathcal{L}_{\text{val}} \left(A_\lambda(\mathcal{D}_{\text{train}}) \right)$$

A_λ - алгоритм обучения с гиперпараметрами λ; на практике внешнее среднее берут кросс-валидацией:

$$\lambda^* = \arg\min_{\lambda \in \Lambda} \mathbb{E}_{(\mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{val}})} \left[\mathcal{L}_{\text{val}}(A_\lambda) \right]$$

Дорого

Одно значение f(λ) – это целое обучение модели. Тысячи вычислений недопустимы.

Без градиента

По λ нельзя продифференцировать функционал. Классическая оптимизация неприменима.

С шумом

Случайность инициализации и разбиений делает f(λ) стохастической.

Grid search и проклятие размерности

Сетка по каждому из d параметров с k значениями требует полного перебора комбинаций:

$$N = k^d$$

Почему сетка плоха

- 5 параметров по 5 значений - уже 3125 обучений
- Добавление одного параметра умножает стоимость на k
- Сетка тратит бюджет на неважные параметры: каждое значение важного повторяется k^{d-1} раз
- Разрешение по каждой оси остаётся грубым

Ключевое наблюдение

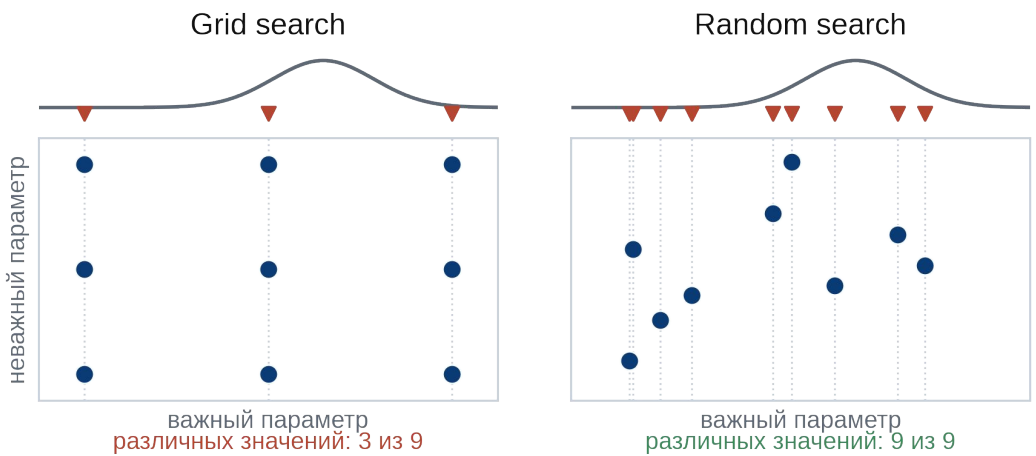
В реальных задачах важны обычно 2–3 параметра из десятка (learning rate, глубина, регуляризация). Остальные почти не влияют на результат.

Эффективная размерность задачи низкая – но сетка об этом не знает и платит за все d осей одинаково.

↓

Отсюда идея дальше: тот же бюджет, потраченный случайно, даёт важным параметрам больше значений.

Случайный поиск: те же затраты, лучшее покрытие



При одинаковом числе точек случайный поиск даёт по важному параметру столько же различных значений, сколько всего испытаний, а сетка - лишь k.

Правило:

Вероятность попасть хотя бы одной точкой в лучшие 5% пространства:

$$1 - 0.95^{60} \approx 0.95$$

Что это правило даёт на практике

- 60 случайных точек - и с вероятностью ~95% одна в топ-5%
- Число не зависит от размерности пространства
- Легко распараллелить: точки независимы
- Baseline, ниже которого HPO опускаться не должен

Формула предполагает независимые равномерные испытания; «5%» - доля объёма пространства, а не близость к оптимуму.

Часть 2. Байесовская оптимизация: учимся на прошлых испытаниях

Идея: суррогатная модель плюс функция полезности

Строим дешёвую вероятностную модель дорогой функции f и по ней выбираем, где пробовать дальше:

$$\lambda_{t+1} = \arg\max_{\lambda} \alpha(\lambda;\ \mathcal{M}_t)$$

1 Суррогат

По накопленным (λ, f) строим модель M□, дающую прогноз μ и неопределённость σ в любой точке.

2 Полезность

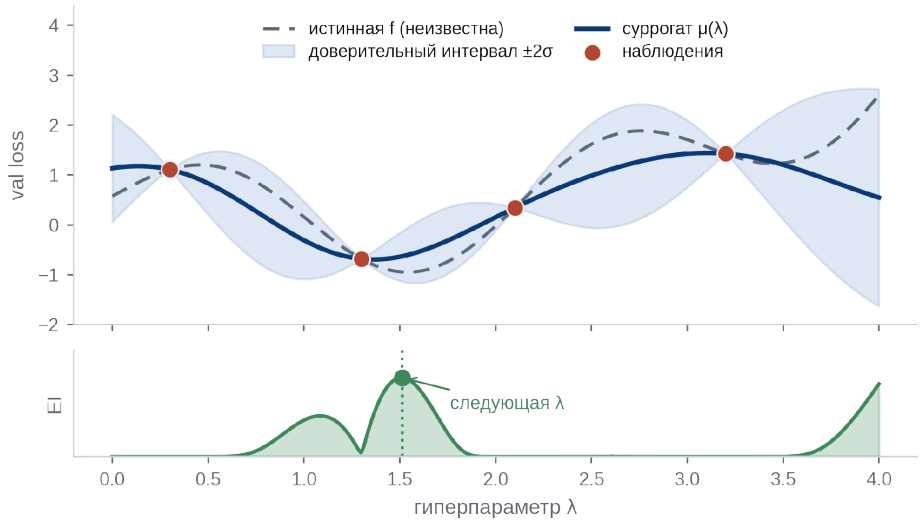
Acquisition-функция α превращает μ и σ в число «насколько стоит пробовать здесь».
Следующей проверяем ту конфигурацию, которая максимизирует полезность по текущей модели

3 Оценка

Считаем настоящую f там, где α максимальна, добавляем в историю и повторяем.

Оптимизация α сама по себе дешёвая – она делается по суррогату, а не по настоящей модели.

Как это выглядит: суррогат и Expected Improvement



Что показывает картинка

- Синяя линия - прогноз суррогата, полоса - его неопределённость ±2σ
- В точках наблюдений неопределённость нулевая; между ними - растёт
- EI внизу велика там, где прогноз лосса низкий (exploitation) ИЛИ разброс большой (exploration)
- Следующая λ - в максимуме EI: метод сам балансирует эти два стремления

Гауссовский процесс и выбор acquisition-функции

Gaussian Process как суррогат

$$f \sim \mathcal{GP}(m(\lambda),\, k(\lambda, \lambda'))$$

Апостериорно в каждой точке - нормальное распределение:

$$f(\lambda) \,|\, \mathcal{D}_t \;\sim\; \mathcal{N}(\mu_t(\lambda),\, \sigma_t^2(\lambda))$$

- Даёт и прогноз μ, и честную неопределённость σ
- Ядро k задаёт гладкость функции
- Точен при малом числе испытаний
- Плохо масштабируется: O(t³) по числу точек и слаб при многих измерениях

Три типичные acquisition-функции

Expected Improvement

$$\mathrm{EI}(\lambda) = \sigma_t(\lambda)[z\,\Phi(z) + \phi(z)], \quad z = \frac{f^* - \mu_t(\lambda)}{\sigma_t(\lambda)}$$

ожидаемая величина улучшения над текущим лучшим; самая ходовая

Lower Confidence Bound

$$\mathrm{LCB}(\lambda) = \mu_t(\lambda) - \kappa\,\sigma_t(\lambda)$$

оптимизм: к напрямую регулирует тягу к разведке

Probability of Improvement

вероятность любого улучшения; слишком жадная, разведывает мало — на практике почти не используется

Φ и φ – функция и плотность стандартного нормального распределения; f - лучшее значение, найденное к шагу t.*

TPE: подход, который масштабируется лучше GP

Tree-structured Parzen Estimator моделирует не f по λ, а наоборот — распределение λ при разном уровне качества. Порог y* делит испытания на «хорошие» и «плохие»:

$$\ell(\lambda) = p(\lambda \mid y < y^*) \qquad g(\lambda) = p(\lambda \mid y \geq y^*)$$

Правило выбора

Максимизация Expected Improvement сводится к простому отношению плотностей:

$$\mathrm{EI} \propto \left(\gamma + \frac{g(\lambda)}{\ell(\lambda)}(1-\gamma)\right)^{-1} \Rightarrow \max_{\lambda} \frac{\ell(\lambda)}{g(\lambda)}$$

Пробуем там, где параметры чаще давали хороший результат и реже — плохой.

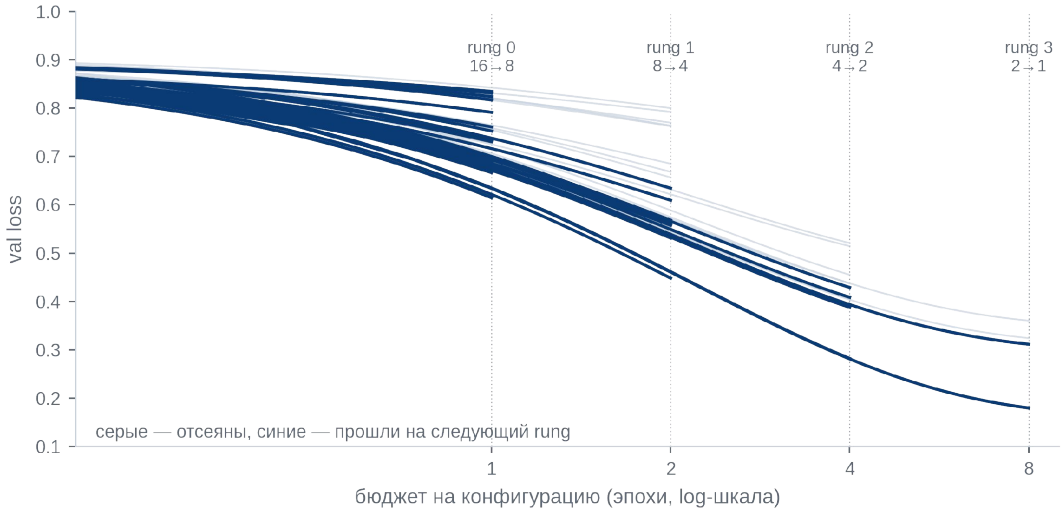
Почему это удобно на практике

- Стоимость линейна по числу испытаний, а не кубична
- Естественно работает с категориальными и условными параметрами
- Дерево описывает зависимости: «этот параметр важен только если выбран тот оптимизатор»
- Это дефолтный сэмплер в Optuna

Bergstra et al. (2011). γ - доля испытаний, относимых к «хорошим» (обычно порядка 15–25%).

Часть 3. Multi-fidelity: не доводить безнадёжные конфигурации до конца

Successive Halving: делим бюджет по этапам



Начинаем много конфигураций на очень маленьком бюджете, оставляем лучшую долю, удваиваем им бюджет - и так до конца.

Расписание бюджета

$$r_i = r_{\min} \cdot \eta^i, \qquad n_i = \lfloor n_0 \, \eta^{-i} \rfloor$$

$$n_{i+1} = \lfloor n_i / \eta \rfloor$$

Почему это работает

- Плохие конфигурации отсеиваются на дешевом бюджете
- Полный бюджет тратится только на выживших
- η задает агрессивность отбора (обычно 3 или 4)
- Работает с любым частичным обучением: эпохи, деревья, доля данных

Дилемма n и B, и как её решает Hyperband

При фиксированном общем бюджете B приходится выбирать: много конфигураций на малом бюджете каждой или мало - на большом.

Много конфигураций, малый бюджет

Широкий обзор, но ранний отбор врёт: конфигурации с медленным стартом отсеиваются напрасно.

Мало конфигураций, большой бюджет

Каждую оцениваем честно, но рискуем вообще не попасть в хорошую область пространства.

Hyperband: перебрать сам компромисс

Вместо угадывания одного расписания Hyperband запускает несколько «скобок» Successive Halving с разными стартовыми точками - от агрессивной до консервативной - и распределяет между ними общий бюджет.

$$s_{\max} = \lfloor \log_\eta (r_{\max}/r_{\min}) \rfloor, \quad B = (s_{\max}+1)\, r_{\max}$$

Одна скобка (bracket в статье ~ вариант) - почти random search, другая - агрессивный отбор; хотя бы одна близка к оптимуму.

ASHA: асинхронная версия для параллельных ресурсов

Классический Successive Halving синхронен: чтобы отобрать лучших на rung, надо дождаться всех. При параллельном запуске это простаивающие воркеры.

Проблема синхронности

- Rung не закрывается, пока не досчитаны все конфигурации этапа
- Один медленный trial держит весь пул воркеров
- На кластере из сотен машин это огромные потери

Решение ASHA

Как только конфигурация готова и входит в лучшую долю уже завершённых на своём rung - сразу продвигаем её выше, не дожидаясь остальных:

$$\text{promote if } \text{rank}(\lambda) \leq \lfloor |\text{rung}_k| / \eta \rfloor$$

- Воркеры никогда не простаивают
- Ошибки раннего продвижения исправляются потоком новых испытаний
- Стандарт для распределённого HPO (Ray Tune, Optuna)

Часть 4. Early stopping и регуляризация как часть поиска

Early stopping: два разных смысла одного термина

Внутри обучения одной модели

$$\hat{t} = \arg\min_{t} \mathcal{L}_{\text{val}}(t)$$

- Останавливаем добавление итераций, когда ошибка на валидации перестала падать
- Число итераций - это гиперпараметр, который подбирается бесплатно, по ходу обучения
- В GBDT - параметр early_stopping_rounds; в нейросетях - patience
- Валидация для остановки должна быть отделена от тестовой выборки

Внутри поиска гиперпараметров

- Прекращаем целое испытание, если промежуточное качество явно хуже уже виденных
- Это ровно то, что делают Successive Halving и median pruning
- Экономит стоимость каждого обращения к f, а не их число
- Риск тот же, что у SHA: медленный старт можно оборвать напрасно

**Оба смысла - про то, чтобы не тратить ресурс на заведомо не улучшающуюся кривую. Разница в том, что именно останавливается: рост одной модели или всё испытание.*

Почему ранняя остановка - это регуляризация

Ограничивая число шагов оптимизации, мы ограничиваем, насколько далеко параметры уходят от начала. Для линейной модели с квадратичным loss это можно показать точно:

Явная L2-регуляризация

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2$$

Штраф за большие веса стягивает решение к нулю; λ задаёт силу стягивания.

Ранняя остановка градиентного спуска

Число шагов, умноженное на learning rate, играет роль обратной силы регуляризации:

$$t_{\text{stop}} \cdot \varepsilon \approx \frac{1}{\lambda}$$

Мало шагов ≈ сильная регуляризация; много шагов ≈ слабая. Остановка по валидации выбирает эту силу автоматически.

Что именно тюнить: регуляризация в GBDT

Ёмкость дерева

- max_depth или num_leaves - главный рычаг сложности
- min_child_weight - минимальная сумма гессианов в листе
- min_data_in_leaf - минимум объектов в листе

Сжатие и объём

- learning_rate вместе с числом деревьев
- early_stopping_rounds - число деревьев по валидации
- subsample и colsample - стохастичность

Явные штрафы

- reg_lambda (L2) на значения листьев
- reg_alpha (L1) на значения листьев
- gamma - минимальный выигрыш для разбиения

Все три группы делают одно - ограничивают переобучение - и потому взаимозаменяемы. Тюнить стоит по одному представителю из группы, а не все сразу.

Часть 5. Optuna и трекинг экспериментов

Optuna: define-by-run и прунинг

Пространство поиска описывается прямо в коде обучения: параметр появляется в тот момент, когда его запрашивают. Это позволяет условные и динамические пространства.

$$\lambda \sim \text{trial.suggest}(\cdot) \text{ — пространство строится по ходу}$$

Что даёт Optuna из коробки

- Сэмплеры: TPE по умолчанию, а также CmaEs, GP и обычный random
- Прунеры: median pruning, Successive Halving, Hyperband
- study.optimize связывает сэмплер и прунер в один цикл
- Хранение испытаний в БД: параллельный и возобновляемый поиск

Median pruning

Простой и надёжный дефолт: обрываем испытание, если его промежуточный результат хуже медианы предыдущих на том же шаге.

$$\text{prune if } \mathcal{L}_t(\lambda) > \text{median}\{\mathcal{L}_t(\lambda_j)\}_{j<t}$$

Требует, чтобы модель отдавала промежуточную метрику через report — то есть поддерживала «частичное» обучение.

Сэмплер отвечает за то, ГДЕ пробовать; прунер - за то, КОГДА прекратить. Это ортогональные решения, их комбинируют.

Трекинг экспериментов

Что теряется без трекинга

- Лучшие параметры найдены, но неизвестно, на какой версии данных и кода
- Результат нельзя воспроизвести: seed, окружение и препроцессинг не сохранены
- Сравнение запусков идёт по памяти и разрозненным файлам
- При параллельном поиске легко перезаписать чужой результат

Что логировать на каждое испытание

- Гиперпараметры и итоговую метрику
- Промежуточные кривые обучения (для прунинга и диагностики)
- Версию данных, кода (git-хеш) и random seed
- Артефакты: модель, конфиг, окружение

Источники

1. Bergstra, J., Bengio, Y. (2012). “Random Search for Hyper-Parameter Optimization.” JMLR 13.
2. Bergstra, J. et al. (2011). “Algorithms for Hyper-Parameter Optimization.” NeurIPS 24 (TPE).
3. Snoek, J., Larochelle, H., Adams, R. (2012). “Practical Bayesian Optimization of Machine Learning Algorithms.” NeurIPS 25.
4. Shahriari, B. et al. (2016). “Taking the Human Out of the Loop: A Review of Bayesian Optimization.” Proc. IEEE.
5. Jamieson, K., Talwalkar, A. (2016). “Non-stochastic Best Arm Identification and Hyperparameter Optimization.” AISTATS.
6. Li, L. et al. (2017). “Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization.” JMLR 18.
7. Feurer, M., Hutter, F. (2019). “Hyperparameter Optimization.” In: Automated Machine Learning, Springer.